

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна

Студентка гр. 4345

Савонькина Е.Н.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм поиска расстояния Левенштейна.
Провести исследование его работы на различных размерах входных данных.

Основные теоретические положения.

Расстояние Левенштейна — это мера различия между двумя строками. Оно показывает минимальное количество операций редактирования, необходимых для превращения одной строки в другую.

Разрешённые операции редактирования:

- вставка символа
- удаление символа
- замена одного символа на другой

В классическом варианте стоимость всех операций равна 1, а если символы совпадают, стоимость замены равна 0.

Основная рекуррентная формула:

$$D(i, j) = \begin{cases} 0, & i = 0, j = 0 \\ i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ \min\{ \\ \quad D(i, j - 1) + 1, \\ \quad D(i - 1, j) + 1, \\ \quad D(i - 1, j - 1) + m(S_1[i], S_2[j]) \\ \} \end{cases}, \quad j > 0, i > 0$$

После заполнения всей матрицы значение в правой нижней ячейке является расстоянием Левенштейна между строками.

Задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

1. Сначала нужно совершить четыре операции удаления символа:
`pedestal` -> `stal`.
2. Затем необходимо заменить два последних символа: `stal` -> `stie`.
3. Потом нужно добавить символ в конец строки: `stie` -> `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Sample Input:

`pedestal`

`stien`

Sample Outut:

7

Вариант 1.

"Особо заменяемый символ и особо добавляемый символ": цена замены определённого символа отличается от обычной цены замены; цена добавления другого (или того же) определённого символа отличается от обычной цены добавления. Особо заменяемый символ и цена его замены, особо добавляемый символ и цена его добавления — дополнительные входные данные.

Основная идея рекурсивного алгоритма.

Выполнение алгоритма расстояния Левенштейна начинается с ввода двух строк, которые необходимо сравнить. Первая строка считается исходной, а вторая — целевой. Необходимо определить минимальную стоимость преобразования первой строки во вторую.

Сначала определяется длина обеих строк. После этого создаётся матрица размеров (длина первой строки + 1) на (длина второй строки + 1). Эта матрица будет хранить минимальные стоимости преобразования подстрок.

Далее заполняются начальные значения матрицы. Первая строка заполняется значениями стоимости последовательных вставок символов второй строки, так как пустую строку можно превратить в подстроку второй строки только вставками. Первый столбец заполняется стоимостью последовательных удалений символов первой строки, так как подстроку первой строки можно превратить в пустую строку только удалением символов.

После инициализации начинается заполнение всей матрицы. Для каждой ячейки с координатами i и j рассматриваются три возможные операции.

После заполнения всей матрицы искомое расстояние Левенштейна находится в правой нижней ячейке. Это значение показывает минимальную суммарную стоимость операций вставки, удаления и замены, необходимых для преобразования первой строки во вторую.

Модификация.

Модификация алгоритма заключается в изменении стоимости некоторых операций. В программе задаются два дополнительных параметра: символ с особой стоимостью вставки и символ с особой стоимостью замены. Также задаются значения этих стоимостей.

Для реализации особой стоимости вставки используется *def ins_cost(a)*. Она принимает символ, который необходимо вставить. Если этот символ совпадает с заданным особым символом вставки, функция возвращает специальную стоимость вставки. В противном случае возвращается стандартная стоимость вставки, равная единице.

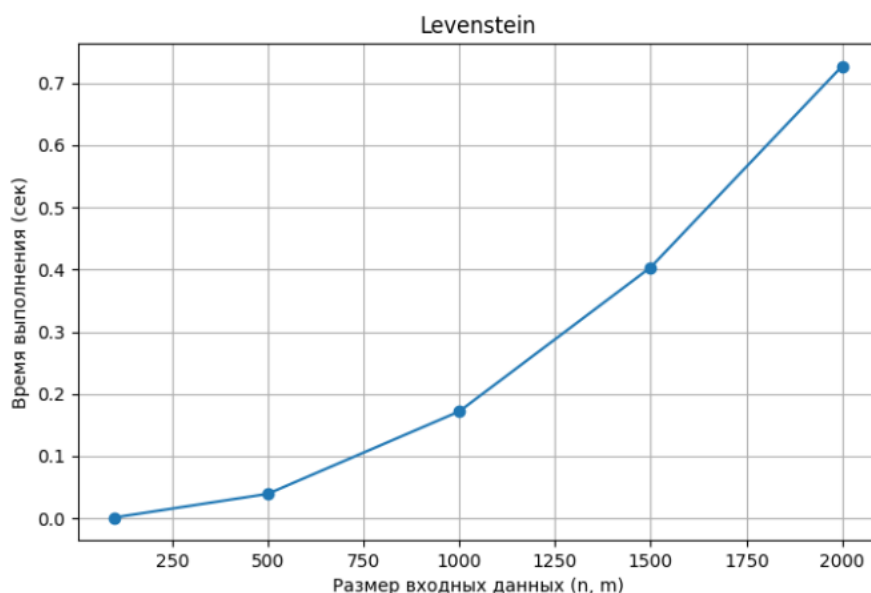
Особая стоимость замены реализуется аналогичным образом.

Кроме того, модификация влияет на инициализацию первой строки матрицы. Поскольку она соответствует последовательным вставкам символов второй строки, значения вычисляются накоплением стоимости вставок для

каждого конкретного символа с использованием функции определения стоимости вставки.

Оценка сложности.

Временная сложность алгоритма определяется количеством вычислений при заполнении матрицы. Для каждой ячейки матрицы выполняется постоянное количество операций: вычисление стоимости удаления, вставки и замены, и выбор минимального значения. Поскольку матрица содержит $(n + 1) * (m + 1)$ ячеек, время работы алгоритма пропорционально произведению длин строк. Таким образом, временная сложность алгоритма равна $O(n * m)$.



Пространственная сложность определяется объёмом памяти, необходимым для хранения матрицы динамического программирования. Матрица содержит $(n + 1) * (m + 1)$ элементов, поэтому используемая память также пропорциональна произведению длин строк. Пространственная сложность алгоритма составляет $O(n * m)$.

Тестирование.

Таблица тестирования для корректных значений:

Строки	Результат	Комментарии
а	1	Результат корректен.
а	1	Результат корректен.

aaa bbb	3	Результат корректен.
a aaa a 2	4	Результат корректен.
a b a 5	2	Результат корректен.

Вывод.

В ходе работы был изучен алгоритм расстояния Левенштейна и реализован. В программу была добавлена модификация, позволяющая задавать особую стоимость вставки и замены символов. Был проведён анализ сложности алгоритма и экспериментальное исследование времени его работы. Полученные результаты подтвердили теоретическую оценку временной сложности алгоритма.

ПРИЛОЖЕНИЕ А.
ИСХОДНЫЙ КОД ПРОГРАММЫ.

```
def ins_cost(a):
    if sp_ins == a:
        return sp_ins_cost
    return 1

def rep_cost(a, b):
    if sp_rep == a:
        return sp_rep_cost
    if a == b:
        return 0
    return 1

def levenshtein(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for i in range(n + 1)]
    dp[0][0] = 0

    print("Матрица до заполнения:")

    print("  ", end=" ")
    for ch in b:
        print(ch, end=" ")
    print()

    for i in range(len(dp)):
        if i == 0:
            print(" ", end=" ")
        else:
            print(a[i - 1], end=" ")

        for j in range(len(dp[0])):
            print(dp[i][j], end=" ")
        print()

    for i in range(n + 1):
        dp[i][0] = i

    for j in range(1, m + 1):
        dp[0][j] = dp[0][j - 1] + ins_cost(b[j - 1])
```

```

print("Матрица после заполнения начальных строк:")

print("  ", end=" ")
for ch in b:
    print(ch, end=" ")
print()

for i in range(len(dp)):
    if i == 0:
        print(" ", end=" ")
    else:
        print(a[i - 1], end=" ")

    for j in range(len(dp[0])):
        print(dp[i][j], end=" ")
    print()

for i in range(1, n + 1):
    for j in range(1, m + 1):
        print(f"Рассматриваем клетку: ({i}, {j})")
        print("Возможные операции и их итоговая стоимость:")

        delete_cost = dp[i - 1][j] + 1
        print(f"Удаление '{a[i - 1]}': {dp[i - 1][j]} + 1 = {delete_cost}")

        insert_cost = dp[i][j - 1] + ins_cost(b[j - 1])
        print(f"Вставка '{b[j - 1]}': {dp[i][j - 1]} + {ins_cost(b[j - 1])} = {insert_cost}")

        replace_cost = dp[i - 1][j - 1] + rep_cost(a[i - 1], b[j - 1])
        print(
            f"Замена '{a[i - 1]}' -> '{b[j - 1]}': {dp[i - 1][j - 1]} + {rep_cost(a[i - 1], b[j - 1])} = {replace_cost}")

        dp[i][j] = min(delete_cost, insert_cost, replace_cost)

        print(f"Выбрано минимальное значение: {dp[i][j]}")

print("Матрица после заполнения:")

print("  ", end=" ")
for ch in b:

```



```

        print(ch, end=" ")
    print()

    for i in range(len(dp)):
        if i == 0:
            print(" ", end=" ")
        else:
            print(a[i - 1], end=" ")

        for j in range(len(dp[0])):
            print(dp[i][j], end=" ")
        print()

    return dp[n][m]

a = input()
b = input()

sp_ins = input("Оособо добавляемый: ")
if sp_ins:
    sp_ins_cost = int(input("Цена: "))
else:
    sp_ins = None
    sp_ins_cost = 1

sp_rep = input("Оособо заменяемый: ")
if sp_rep:
    sp_rep_cost = int(input("Цена: "))
else:
    sp_rep = None
    sp_rep_cost = 1

print(levenshtein(a, b))

```